CSC3002
C/C++ Programming

Dynamic Memory Management

Prof. Dr. Ruizhi Chen，SDS
Office: TxC 717

Dynamic Memory Management

1 Dynamic allocation and the heap
2 Freeing memory
3 Heap-stack diagrams
4 Copying objects
5 The uses of const
6 Smart Pointer

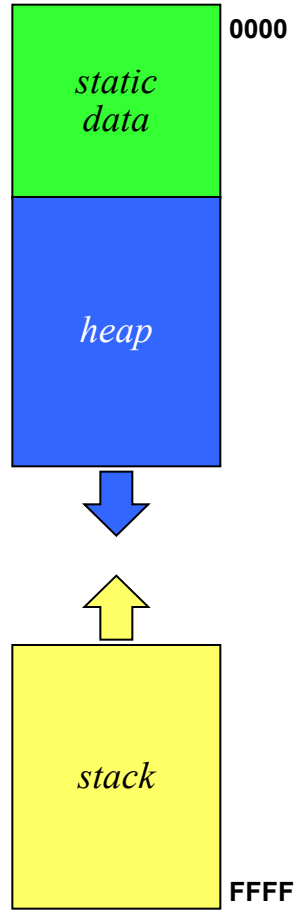
Pointers

- In C++, pointers serve several purposes, of which the following are the most important:
  - Pointers allow you to refer to a large data structure in a compact way. Because a memory address typically fits in a few bytes of memory, this strategy offers considerable space savings when the data structures themselves are large. E.g., call by pointers.
  - Pointers make it possible to reserve new memory during program execution. In many applications, it is convenient to acquire new memory as the program runs and to refer to that memory using pointers, which is called dynamic allocation.
  - Pointers can be used to record relationships among data items. Data structures that use pointers to create connections between individual components are called linked structures. Programmers can indicate that one data item follows another in a conceptual sequence by including a pointer to the second item in the internal representation of the first.

The Allocation of Memory to Variables



- When you declare a variable in a program, C++ allocates space for that variable from one of several memory regions.
- One region of memory is reserved for program code and global variables/constants that persist throughout the lifetime of the program. This information is called static data.
- Each time you call a method, C++ allocates a new block of memory called a stack frame to hold its local variables. These stack frames come from a region of memory called the stack.
- It is also possible to allocate memory dynamically, as we will describe in Chapter 12. This space comes from a pool of memory called the heap.
- In classical architectures, the stack and heap grow toward each other to maximize the available space.



Dynamic Allocation

- C++ uses the new operator to allocate memory on the heap.
- You can allocate a single value (as opposed to an array) by writing new followed by the type name. Thus, to allocate space for a int on the heap, you would write:

```
int *pi = new int;
```

- You can allocate an array of values using the following form:

```
type *name = new type[size];
```

Thus, to allocate an array of 10000 integers, you would write:

```
int *arr = new int[10000];
```

- The delete operator frees memory previously allocated. For arrays, you need to include empty brackets, as in:

```
delete pi;
delete [] arr;
```

Using **delete**

- Always pair a delete to a new, and a delete [] to a new [].
- No size needs to be specified for delete []. One of the approaches for compilers know how much memory to free is to allocate a little more memory and to store a count of the elements in a head segment (invisible to you) just before the first array element.
- delete ptr only frees the memory space pointed by ptr, but not the memory space occupied by ptr itself. ptr is still a live variable until it is released (depending on how it is declared), though it is dangling (i.e., does not point to a valid object of the appropriate type).
- To avoid dangling pointers, after deleting a pointer (freeing the memory space it points to), one can nullify a pointer by:

```
ptr = NULL; // nullptr since C++11
```

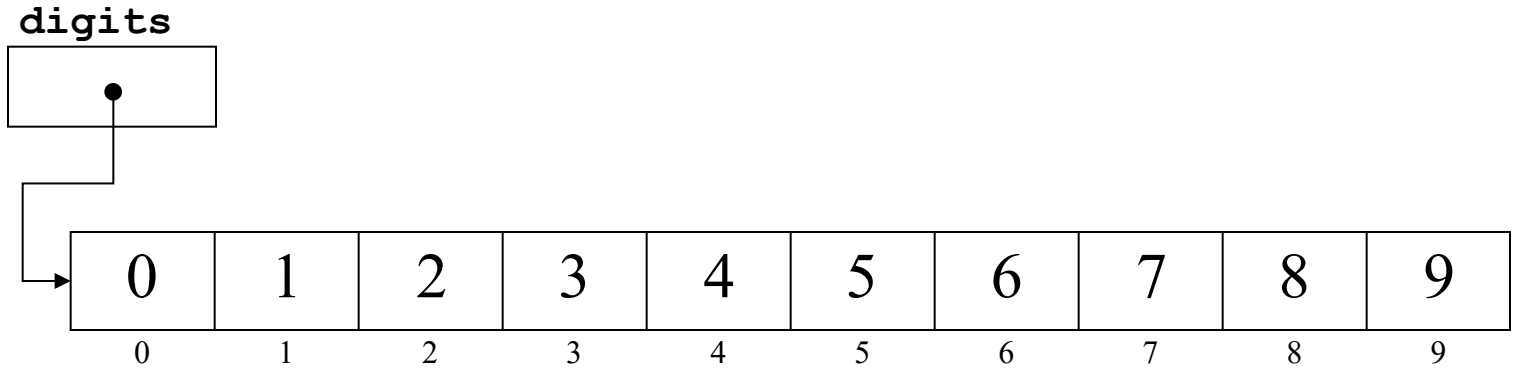
But there might be other pointers pointing to the same address that are not nullified.

Dynamic Arrays

- Write a method createIndexArray(n) that returns an integer array of size n in which each element is initialized to its index. As an example, calling

```
Int *digits = createIndexArray(10);
```

should result in the following configuration:



- How would you free the memory allocated by this call?

Dynamic Arrays

```
/*
 * Function: createIndexArray
 * Usage: int *array = createIndexArray(n);
 * ----------------------------------------
 * Returns a new dynamic integer array of length n
 * in which every element is set to its own index.
 */

int* createIndexArray(int n) {
   int *array = new int[n];
   for (int i = 0; i < n; i++) {
      array[i] = i;
   }
   return array;
}

int main() {
   int *digits = createIndexArray(10);
   delete [] digits;
}
```

If we change dynamic array to automatic: `int array[n];` *what happens?*

If we change `10` *to* `1`*, do we still need* `[]` *after* `delete`*?* *Yes.*

Dynamic 2D Arrays

```
int **CreateMatrix(int rows, int cols)
{
        int **array = new int *[rows]; // allocate row pointers, an array of int*
        for (int i = 0; i < rows; i++)
        array[i] = new int[cols]; // allocate each row, an array of int
        return array;
}

void DeleteMatrix(int **arr, int rows)
{
        for (int i = 0; i < rows; i++)
                delete[] arr[i]; // delete each row, array of int
        delete[] arr; // delete the pointers of rows
}

int main()
{
        int rows = 10;
        int cols = 5;
        int **arr = CreateMatrix(rows, cols);
        for (int i = 0; i < rows; i++)
        {
                for (int j = 0; j < cols; j++)
                                arr[i][j] = i + j;
        }
        cout << "arr[3][4]=" << arr[3][4] << endl;
        DeleteMatrix(arr, rows);
}
```

Dynamic 2D Arrays – Implement it as an object

```
#pragma once

class matrix
{
public:
	int **pData = nullptr;
	int rows = 0;
	int cols = 0;
	matrix(int r = 0, int c = 0);
	void create();
	int &operator()(int i, int j) { return pData[i][j]; }
	~matrix();
};
```

Dynamic 2D Arrays – Implement it as an object

```
#include "matrix.h"

matrix::matrix(int row, int col)
{
        rows = row;
        cols = col;
        create();
}

matrix::~matrix()
{
        if (!pData)
        return;
        for (int i = 0; i < rows; i++)delete[] pData[i]; // delete each row, array of int
        delete[] pData;                                  // delete the pointers of rows
}


void matrix::create()
{
        pData = new int *[rows]; // allocate row pointers, an array of int*
        for (int i = 0; i < rows; i++)
        pData[i] = new int[cols]; // allocate each row, an array of int
}
```

RAII – Resource Acquisition Is Initialization



It is a c++ programming idiom where resource management is tiled to object lifetime. Acquire resources in constructor, release them in destructor.

Dynamic 2D Arrays – Implement it as an object

```
#include <iostream>
#include "matrix.h"
using namespace std;

int main()
{
    matrix m(5, 5);
    m(3, 4) = m(2, 1) = 100;
    cout << "m(2,1)=" << m(2, 1) << endl;
    cout << "m(3,4)=" << m(3, 4) << endl;
}
```



```
m(2,1)=100
m(3,4)=100
```

RAII – Resource Acquisition Is Initialization



It is a c++ programming idiom where resource management is tiled to object lifetime. Acquire resources in constructor, release them in destructor.

Dynamic 2D Arrays – Another Simple Implementation

```
class Matrix
{
private:
        int *pData = nullptr;
        void create() { if (!pData) pData = new int[rows * cols];}

public:
        int rows = 0;
        int cols = 0;
        Matrix(int r = 0, int c = 0) : rows(r), cols(c) { create(); }
        int &operator()(int i, int j) { return pData[i * cols + j]; }
        ~Matrix(){ if (pData) delete[] pData;}
};



void Test_Matrix()
{
        Matrix m(5, 5);
        m(3, 4) = m(2, 1) = 1000;
        Std::cout << "m(2,1)=" << m(2, 1) << std::endl;
        Std::cout << "m(3,4)=" << m(3, 4) << std::endl;
}
```



```
m(2,1)=1000;
m(3,4)=1000;
```

Garbage collection

- The big challenge in working with dynamic memory allocation is freeing the heap memory you allocate. Programs that fail to do so have what computer scientists call memory leaks.
- In Python and Java, objects are created on the heap and are automatically reclaimed by a garbage collector when those objects are no longer accessible. This discipline makes memory management very easy and convenient (for you).
- Garbage collection frees the programmer from manually dealing with memory deallocation. As a result, certain categories of bugs are eliminated or substantially reduced:
  - Certain kinds of memory leaks;
  - Dangling pointer bugs;
  - Double free bugs;
  - Efficient implementations of persistent data structures, etc.

```
char *p1 = new char;
char *p2 = new char;
p2 = p1;
delete p1;
p1 = NULL;
delete p2; // double free bug
```

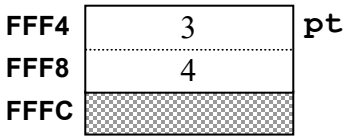
C++ Memory Management

- Garbage collection, however, is not always efficient:
  - Consuming additional resources;
  - Performance impacts;
  - Possible stalls in program execution;
  - Incompatibility with manual resource management, etc.
- For performance, garbage collection is not adopted in C++:
  - Objects can be allocated either on the stack or in the heap.
  - Programmers must manage heap memory allocation explicitly.
- In object-oriented programming, it is nearly impossible for clients to remember exactly what heap storage is currently active when using objects. Fortunately, the designers of C++ made it possible to free heap memory allocated by objects (clients) when those objects go out of scope on the stack (e.g. RAII).
- In well-designed C++ programs, (the implementer of) each class takes responsibility for its own heap storage (RAII).

Allocating a **`Point`** Object

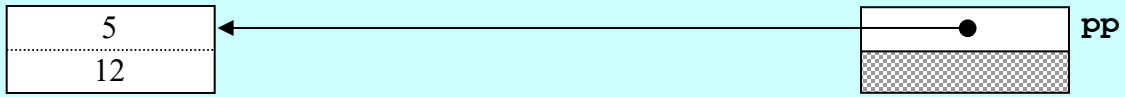
- The usual way to allocate a Point object is to declare it as a local variable on the stack, as follows:

```
Point pt(3, 4);
```



- It is, however, also possible to allocate a Point object on the heap using the following code:

```
Point* pp = new Point(5, 12);
```



- However, because of the data protection mechanism, it is nearly impossible for clients to know exactly what heap storage is currently active when deleting objects. For instance, some of the data fields might have been using the heap memory dynamically allocated by the member methods.

Destructors

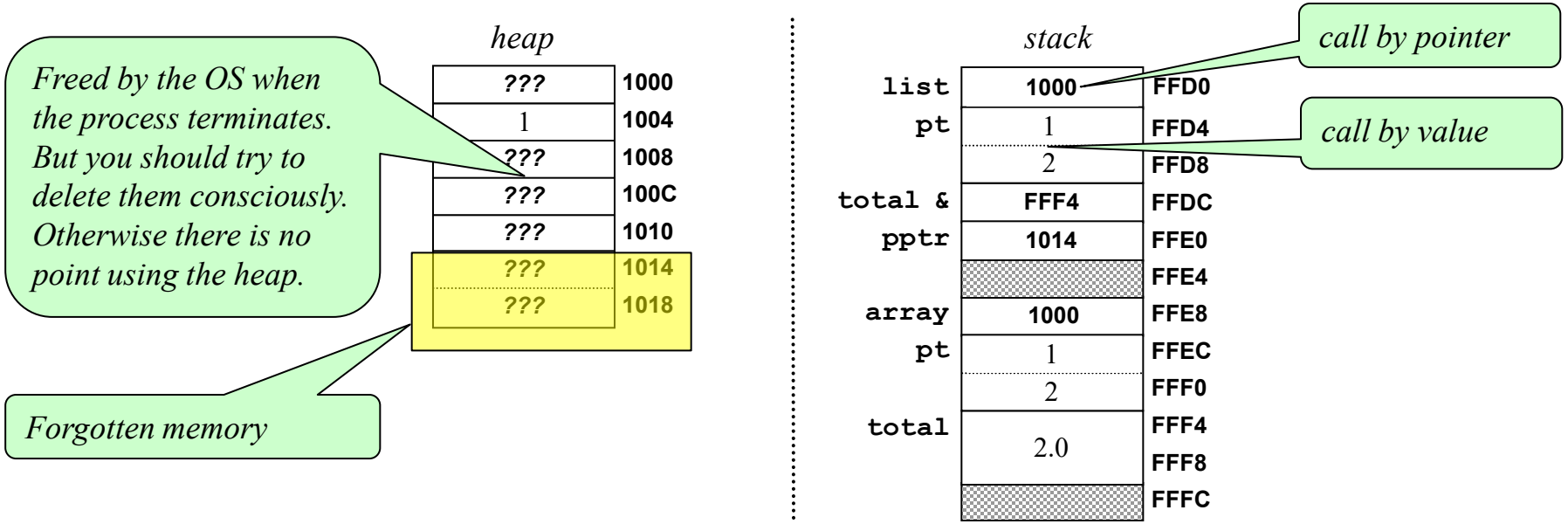
- In C++, class definitions often include a destructor, which specifies how to free the storage used to represent an instance of that class.
- The prototype for a destructor has no return type and uses the name of the class preceded by a tilde (~). The destructor must not take any arguments.
- C++ calls the destructor automatically whenever a variable of a particular class is released. For stack objects, this happens when the function returns. The effect of this rule is that a C++ program that declares its objects as local variables on the stack will automatically reclaim those variables.
- If you instead allocate space for an object in the heap using new, you must explicitly free that object by calling delete. Calling delete also automatically invokes the destructor.
- If new is used in constructors, delete should most probably be used in destructors too.

Heap-Stack Diagrams

- It is easier to understand how C++ works if you have a good mental model of its use of memory. One of the most useful models is a heap-stack diagram, which shows the heap on the left and the stack on the right, separated by a dotted line.
- Whenever your program uses new, you need to add a block of memory to the heap side of the diagram. That block must be large enough to store the entire value you're allocating. If the value is a struct or an object type, that block must include space for all the members inside that structure.
- Whenever your program calls a method, you need to create a new stack frame by adding a block of memory to the stack side. For method calls, you need to add enough space to store the local variables for the method, again with some overhead information that tracks what the program is doing. When a method returns, C++ reclaims the memory in its frame.

Exercise: Heap-Stack Diagrams

```
int main() {
    void nonsense(int list[], Point pt, double & total) {
        Point *pptr = new Point;
        list[1] = pt.x;
        total += pt.y;
    }
```



Dynamic Allocation in C

- Supported in <cstdlib> (stdlib.h)

Dynamic memory management

calloc	Allocate and zero-initialize array (function)
free	Deallocate memory block (function)
malloc	Allocate memory block (function)
realloc	Reallocate memory block (function)

Things that you only need to know that you don't know. I don't need you to memorize it for the exam.

Pointers

- In C++, pointers serve several purposes, of which the following are the most important:
  - Pointers allow you to refer to a large data structure in a compact way. Because a memory address typically fits in a few bytes of memory, this strategy offers considerable space savings when the data structures themselves are large. E.g., call by pointers.
  - Pointers make it possible to reserve new memory during program execution. In many applications, it is convenient to acquire new memory as the program runs and to refer to that memory using pointers, which is called dynamic allocation.
  - Pointers can be used to record relationships among data items. Data structures that use pointers to create connections between individual components are called linked structures. Programmers can indicate that one data item follows another in a conceptual sequence by including a pointer to the second item in the internal representation of the first.

The Uses of const

- The const keyword has many distinct purposes in C++. This text uses it in the following three contexts:
  1. Constant definitions. Adding the keyword const to a variable definition tells the compiler to disallow subsequent assignments to that variable, thereby making it constant.
     ```
     const double PI = 3.14159265358979323846;
     static const int INITIAL_CAPACITY = 10;
     ```
  2. Constant call by reference. Adding const to the declaration of a reference parameter signifies that the function will not change the value of that parameter. This guarantee allows the compiler to share the contents of a data structure without allowing methods to change it.
     ```
     void deepCopy(const CharStack & src);
     ```
  3. Constant methods. Adding const after the parameter list of a method guarantees that the method will not change the object.
     ```
     int CharStack::size() const
     ```
- Classes that use the const specification for all appropriate parameters and methods are said to be const-correct.

Const Class Members

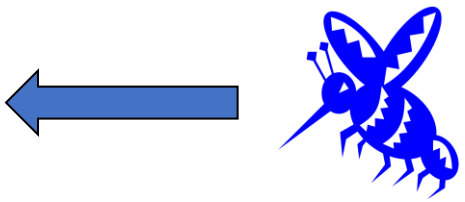
```
class A
{
public:
        const int c1 = 1; // initialize const valuble
        const int c2 = 2;
        // re-initialized c1=x, c2=y
        A(int xx, int yy) : c1(xx), c2(yy), x(xx), y(yy) {};
        int getC1() const { return c1; }
        int getX() const { return x; }
        void add(const int xc) { x += xc; }

        // void setX() const { x = x+1;}

    void setX(int xx) { x = xx; }

private:
        int x;
        int y;
};

int main()
{
        A a(5, 10);
        a.add(100);
        cout << "a.getX() = " << a.getX() << endl;
        cout << "a.getC1() = " << a.getC1() << endl;
        return 0;
}
```



The `random.cpp` Implementation




```
/*
 * Implementation notes: initRandomSeed
 * ------------------------------------
 * The initRandomSeed function declares a static variable that
 * keeps track of whether the seed has been initialized.  The
 * first time initRandomSeed is called, initialized is false,
 * so the seed is set to the current time.
 */

void initRandomSeed() {
   static bool initialized = false;
   if (!initialized) {
      srand(ti me(NULL)));
      initializ e;
   }
}
```

The lifetime of `static` *variables begins the first time the program flow encounters the declaration and it ends at program termination. The compiler allocates only one copy of* `initialized`*, which is initialized exactly once, and then shared by all calls to* `initRandomSeed`*. This ensures that the initialization step must be performed once and only once.*

The Uses of **`static`**

Applied to	Meaning
a local variable	The local static variable is "permanent", in the sense that it is initialized only once and retains its value from one function call to the next. It is like having a global variable with local scope.
a global constant	Since a global static constant has internal linkage by default (unlike global variables, which have external linkage by default), meaning it is only available for use in the file in which it is defined (and is therefore, in effect, static const).
a global variable or a free function	The scope of the static free function, or the static global variable, is limited to the file in which it is defined. Without a static qualifier, any free function or global variable in a file has the extern qualifier by default, making it visible from other files in the compilation unit.
a member variable of a class.	There is only one such static variable for the class, no matter how many objects of the class are created. In other words, it turns the member variable from an "instance variable" into a "class variable".
a member function of a class.	The static member function may access only static members of the class. That is, it may not access any instance members.

Static Class Members

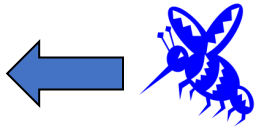
```
class A
{
public:
        // static const, can be define inside the class
        static const int c = 100;
        static int i;
        int k = 0;
        static int f1() { return c; }
        int f2() { return k + c; }
        static int f3() { return i; }
        // static int f4() { return f1() + f2(); }
        static int f4() { return f1() + f3(); }
};

int A::i = 2; // define outside of the class

int main()
{
        A a, b;
        cout << "a.c = " << a.c << " a.i = " << a.i << endl;
        cout << "b.c = " << b.c << " b.i = " << b.i << endl;

        a.i = 3;
        cout << "b.i = " << b.i << endl;

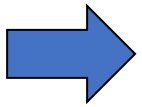
        cout << "a.f3()=" << a.f3() << endl;
        cout << "b.f4()=" << b.f4() << endl;
}
```



Static member – class variable, shared by all instances of the class.

Non-static member – instance variable, only for this instance. Not shared with other instances.

Static function can access only static members of the class.



```
a.c = 100 a.i = 2
b.c = 100 b.i = 2


b.i = 3

a.f3()=3
b.f4()=103
```

Smart Pointer

- In C++, a smart pointer is a class template that wraps a raw pointer and automatically manages its lifetime—typically by deleting the object when it's no longer needed. This helps prevent memory leaks, dangling pointers, and manual delete errors.

Smart Pointer	Purpose
`std::unique_ptr`	Exclusive ownership (preferred default)
`std::shared_ptr`	Shared ownership with reference counting
`std::weak_ptr`	Non-owning observer of `shared_ptr`

Unique Pointer

- By default, an object created using make_unique is allocated on the heap.
- No new/delete pair is needed.
- Memory on heap will automatically freed when pointer goes out of scope.

```
#include <iostream>
#include <memory> // for smart pointer
using namespace std;

void UniquePointer(unsigned long k)
{
    auto p = make_unique<int[]>(k); // create on heap int[k]
    for (int i = 0; i < k; i++) p[i] = i;
    cout << "p[3]=" << p[3] << endl;
}

int main()
{
        UniquePointer(5);
        return 0;
}
```

Shared Pointer

- By default, an object created using make_shared is allocated on the heap.
- The shared pointer holds a number of use counts (co-owners)
- For adding each copy, use counts +1
- For each destruction (out of scope), use counts -1
- Last owner leaves, delete the managed object.

```
#include <iostream>
#include <memory> // for smart pointer
using namespace std;

void SharedPointer(unsigned long k) {
    auto p = make_shared<unsigned long>(k); // use_count = 1;
    if (true) {
        auto p1 = p; // use_count = 2;
        cout << "p1=" << *p1 << " count = " << p.use_count() << endl;
    }
    cout << "p=" << *p << " count = " << p.use_count() << endl;
}

int main() {
        SharedPointer(5);
        return 0;
}
```

Weak Pointer

- A weak pointer is a non-owning observational smart pointer that points to an object managed by a shared pointer without affecting its lifetime.
- The object is deleted when all shared pointers are destructed regardless of how many weak pointer exist.
- Lock the pointer before using the managed object, unlock is not needed.
- Lock() will increments the shared reference count automatically
- The managed object itself is not automatically thread-safe.

```
#include <iostream>
#include <memory> // for smart pointer
using namespace std;

void WeakPointer(int k)
{
     std::shared_ptr<int> sp = make_shared<int>(k);
     std::weak_ptr<int> wp1 = sp;
     std::weak_ptr<int> wp2 = sp;
     std::weak_ptr<int> wp3 = sp;
     auto p = wp3.lock(); // lock it before using it
     cout << "Weak Pointer p=" << *p
     Cout << " count=" << sp.use_count() << endl;
}

int main() {
     WeakPointer(1000);
     return 0;
}
```